

Implementación de Sistemas Digitales con ATmega328P en Lenguaje Ensamblador

Enzo Acosta

Ingeniería Mecatrónica

Universidad Tecnológica (UTEC)

enzo.acosta@estudiantes.utec.edu.uy

Keywords— ATmega328P, lenguaje ensamblador, interrupciones, temporizadores, máquinas de estados.

I. RESUMEN

En el presente laboratorio se desarrollaron diferentes sistemas digitales utilizando el microcontrolador ATmega328P programado en lenguaje ensamblador. Se implementó el control de una lavadora automática mediante una máquina de estados, un contador digital con display de 7 segmentos y un sistema de selección y ejecución de ocho secuencias luminosas. Durante el desarrollo se utilizaron puertos de entrada y salida, interrupciones externas, temporizadores, registros y direccionamiento de memoria. Los programas fueron ensamblados en Microchip Studio y posteriormente simulados en PICSimLab, permitiendo verificar el funcionamiento de las entradas, salidas, temporizaciones y condiciones establecidas para cada ejercicio.

II. INTRODUCCIÓN

La programación de microcontroladores en lenguaje ensamblador permite trabajar directamente con los recursos internos del dispositivo, comprendiendo el funcionamiento de sus registros, memoria, puertos de entrada y salida y periféricos. En esta práctica se utilizó el microcontrolador ATmega328P como plataforma para desarrollar diferentes aplicaciones orientadas al control de sistemas digitales.

El laboratorio se desarrolló mediante la implementación de distintos problemas en lenguaje ensamblador, incluyendo una lavadora automática controlada mediante estados, un contador con display de 7 segmentos y un sistema de secuencias luminosas con ocho LEDs. Para su resolución se utilizaron recursos como interrupciones, temporizadores, operaciones lógicas, direccionamiento de memoria y manejo directo de puertos. Finalmente, los programas fueron simulados en PICSimLab para comprobar su funcionamiento y verificar el cumplimiento de los requerimientos planteados.

III. OBJETIVOS

A. Objetivo General

Desarrollar y analizar sistemas digitales utilizando el microcontrolador ATmega328P programado en lenguaje ensamblador, aplicando los principales recursos de la arquitectura AVR para el control de entradas, salidas y procesos temporizados.

B. Objetivos Específicos

- Implementar diferentes aplicaciones mediante el manejo directo de los puertos de entrada y salida, registros y estructuras de control del ATmega328P.
- Aplicar interrupciones, temporizadores, direccionamiento de memoria y operaciones lógicas para resolver los requerimientos particulares de cada sistema.
- Verificar mediante simulación el correcto funcionamiento de los programas desarrollados, evaluando la respuesta de las entradas, salidas, estados y temporizaciones implementadas.

IV. MATERIALES UTILIZADOS

A. Componentes Electrónicos

- Arduino UNO R3 con microcontrolador ATmega328P.
- LEDs.
- Resistencias.
- Pulsadores.
- Display de 7 segmentos.
- Elementos de entrada y salida representados en simulación para los sensores y motor de la lavadora.

B. Software

- Microchip Studio para el desarrollo y ensamblado de los programas.
- PICSimLab para la simulación y verificación de los sistemas.
- Git y GitHub para el control de versiones y almacenamiento del código.

V. MARCO TEÓRICO

A. Microcontrolador ATmega328P y puertos de entrada/salida

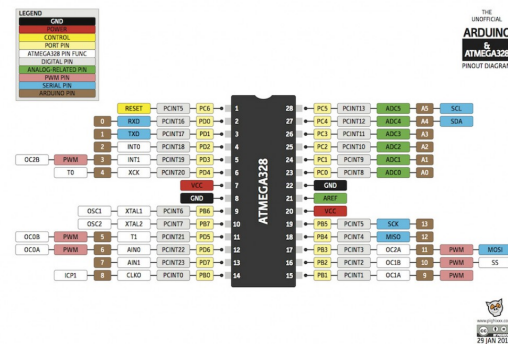


Fig. 1: Distribución de pines del ATmega328P.

El ATmega328P es un microcontrolador de arquitectura AVR que permite controlar directamente diferentes periféricos mediante registros. Para la interacción con elementos externos dispone de los puertos digitales PORTB, PORTC y PORTD, cuyos pines pueden configurarse individualmente como entradas o salidas.

La configuración de los puertos se realiza principalmente mediante los registros DDRx, PORTx y PINx. El registro DDRx determina la dirección de cada pin, donde un valor lógico 1 lo configura como salida y un valor lógico 0 como entrada. PORTx permite establecer el nivel de una salida o activar la resistencia *pull-up* interna cuando

el pin se utiliza como entrada, mientras que PINx permite consultar el estado presente en el puerto.

Estas características permiten conectar directamente al microcontrolador elementos como pulsadores, LEDs, sensores y displays, utilizados como entradas y salidas en los diferentes sistemas desarrollados.

B. Programación en lenguaje ensamblador AVR

El lenguaje ensamblador constituye una capa de programación cercana al hardware, utilizando instrucciones representadas mediante mnemónicos que son posteriormente traducidos a código máquina. Cada familia de microcontroladores dispone de su propio conjunto de instrucciones, permitiendo controlar de forma directa los registros y recursos internos del dispositivo.

En la arquitectura AVR se dispone de registros de propósito general sobre los cuales pueden realizarse operaciones de transferencia, aritméticas, lógicas y de comparación. Instrucciones como LDI y MOV permiten cargar y transferir datos, mientras que IN y OUT permiten interactuar con registros de entrada y salida.

El control del flujo del programa se realiza mediante instrucciones de comparación, saltos y llamadas a subrutinas. De esta forma pueden implementarse estructuras de decisión y repetición necesarias para organizar el comportamiento de los diferentes sistemas.

C. Temporizadores e interrupciones

El ATmega328P dispone de tres temporizadores: Timer0 y Timer2, de 8 bits, y Timer1, de 16 bits. Estos periféricos permiten contar ciclos de reloj o eventos externos y pueden utilizarse para generar referencias temporales sin depender exclusivamente de retardos realizados mediante ciclos de instrucciones.

Los temporizadores poseen registros de configuración y pueden trabajar en diferentes modos de operación. En el modo CTC (*Clear Timer on Compare Match*), el contador se compara con un valor previamente establecido y se reinicia al producirse la coincidencia. Mediante el uso de un prescaler también es posible dividir la frecuencia de reloj aplicada al temporizador para obtener intervalos de tiempo mayores.

Las interrupciones permiten que el microcontrolador responda ante determinados eventos sin realizar una consulta continua de las entradas. Al producirse una interrupción, la ejecución se dirige a una rutina específica y posteriormente continúa con el programa principal. Para su funcionamiento deben habilitarse tanto las interrupciones globales como la fuente de interrupción correspondiente.

Este mecanismo permite atender eventos provenientes de pulsadores y temporizadores, reduciendo la necesidad de monitorear continuamente su estado desde el bucle principal.

D. Memoria SRAM y direccionamiento indirecto

La memoria SRAM del ATmega328P permite almacenar datos que pueden ser modificados durante la ejecución del programa. Además del direccionamiento directo, la arquitectura AVR permite realizar direccionamiento indirecto utilizando los pares de registros X, Y y Z como punteros hacia posiciones de memoria.

Las instrucciones LD y ST permiten leer y escribir datos utilizando estos punteros. También pueden utilizarse variantes con incremento o decremento automático del puntero, facilitando el recorrido de estructuras formadas por varios datos almacenados consecutivamente.

Este mecanismo permite organizar información en memoria y acceder a ella a partir de un índice, evitando utilizar una rutina independiente para cada valor.

E. Tablas de búsqueda

Una tabla de búsqueda o LUT (*Look Up Table*) es una estructura que almacena valores previamente definidos asociados a determinadas entradas. Su uso permite consultar directamente un resultado almacenado en memoria en lugar de realizar repetidamente las operaciones necesarias para calcularlo.

Las LUT permiten reducir la complejidad de determinadas rutinas y mantener una correspondencia directa entre un valor de entrada y su resultado. En sistemas con recursos limitados debe considerarse que este método utiliza espacio de memoria para almacenar los valores de la tabla.

En el caso de un display de 7 segmentos, una LUT puede almacenar el patrón binario correspondiente a cada número. A partir del valor que se desea representar, este puede utilizarse como índice para acceder al patrón correspondiente y posteriormente enviarlo hacia los puertos de salida.

VI. METODOLOGÍA

A. Lavadora automática

Se implementó el control de una lavadora automática utilizando el ATmega328P programado en lenguaje ensamblador. El funcionamiento se organizó mediante una máquina de estados encargada de controlar la selección de carga, los sensores de puerta y llenado, las etapas de lavado, centrifugado y secado, los indicadores LED y el movimiento del motor.

La carga puede seleccionarse entre ligera, media y pesada mediante un único pulsador, manteniéndose indicada durante todo el proceso mediante tres LEDs. Los pulsadores de inicio y selección fueron conectados a las interrupciones externas INT0 e INT1, respectivamente. La selección únicamente puede modificarse mientras el sistema se encuentra en estado de espera.

El Timer0 fue configurado en modo CTC con una interrupción cada 1 ms. Esta referencia se utilizó tanto para el antirrebote de los pulsadores como para generar las unidades de 100 ms empleadas en la temporización de los diferentes estados. De esta forma se evitaban retardos bloqueantes durante la ejecución del proceso.

El sistema verifica inicialmente que la puerta se encuentre cerrada y posteriormente espera la activación del sensor de llenado. Durante el lavado se realizan cinco ciclos de giro y pausa, continuando luego con el centrifugado y finalmente con el secado, compuesto por giro hacia la derecha, pausa y giro hacia la izquierda. Los tiempos de cada etapa se modifican según la carga seleccionada.

Como medida de seguridad, si la puerta se abre durante el proceso el motor se detiene y la temporización queda pausada hasta que la puerta vuelva a cerrarse. La asignación completa de pines, los tiempos y los registros utilizados se presentan en el Apéndice A.

La Fig. 2 presenta la máquina de estados utilizada para representar el funcionamiento general de la lavadora y las condiciones de transición entre cada etapa.

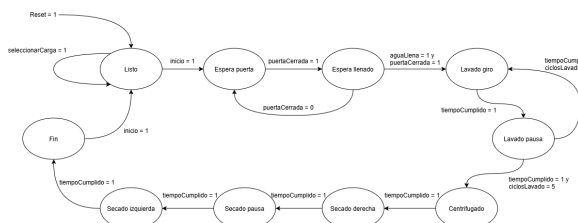


Fig. 2: Máquina de estados de la lavadora automática.

B. Contador digital con display de 7 segmentos

Se implementó un contador digital de un dígito. El sistema permite incrementar, decrementar y reiniciar el valor del contador mediante tres pulsadores, manteniendo el resultado visible de forma permanente en un display de 7 segmentos. Se estableció un rango de funcionamiento entre 0 y 9, evitando incrementos o decrementos fuera de dichos límites.

Los pulsadores de incremento y decremento fueron conectados a las entradas de interrupción externa INT0 e INT1, mientras que el pulsador de reinicio se implementó mediante una interrupción

por cambio de pin. De esta forma, las acciones del usuario son atendidas mediante interrupciones sin requerir una lectura continua de las entradas.

Para evitar múltiples detecciones producidas por el rebote de los pulsadores se utilizó el Timer0 como referencia temporal. El temporizador fue configurado para generar interrupciones periódicas, permitiendo validar la liberación del pulsador antes de aceptar una nueva acción.

La conversión entre el valor del contador y los segmentos a encender se realizó mediante una tabla de búsqueda (LUT). Los patrones correspondientes a los números del 0 al 9 se almacenaron en memoria SRAM y fueron consultados utilizando el registro Y como puntero. Este procedimiento permite obtener directamente el patrón correspondiente al número actual sin realizar comparaciones individuales para cada valor.

Los segmentos A-F del display fueron conectados a PB0-PB5, respectivamente, mientras que el segmento G fue conectado a PD7. La asignación completa de pines y los patrones utilizados en la LUT se presentan en el Apéndice B.

La Fig. 3 presenta la máquina de estados utilizada para representar el flujo general del sistema y las transiciones producidas por los pulsadores.

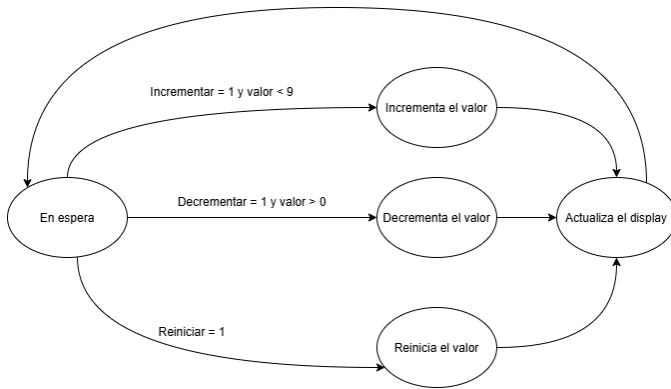


Fig. 3: Máquina de estados del contador digital.

C. Selección y ejecución de secuencias con 8 LEDs

Se implementó un sistema compuesto por ocho LEDs y tres pulsadores, capaz de ejecutar ocho secuencias luminosas diferentes. Los pulsadores permiten avanzar a la secuencia siguiente, retroceder a la anterior o regresar directamente a la secuencia 1. La selección se implementó de forma circular, permitiendo pasar de la secuencia 8 a la 1 y viceversa.

Manteniendo el criterio utilizado en el contador digital, los pulsadores de avance y retroceso fueron conectados a las interrupciones externas INT0 e INT1, mientras que el reinicio se implementó mediante una interrupción por cambio de pin. El Timer0 fue configurado en modo CTC con una interrupción cada 1 ms, utilizándose tanto para el antirrebote de los pulsadores como para controlar el intervalo entre los pasos de las secuencias.

Las secuencias fueron desarrolladas mediante operaciones binarias sobre un patrón de 8 bits. Se utilizaron principalmente instrucciones de desplazamiento, rotación, máscaras y operaciones lógicas, permitiendo generar los diferentes efectos sin recurrir a una lista extensa de patrones constantes. Cada rutina modifica únicamente un paso de la secuencia, evitando retardos bloqueantes y permitiendo cambiar de estado durante la ejecución de una animación.

La asignación completa de pines y la descripción de las ocho secuencias se presentan en el Apéndice C. La Fig. 4 representa la máquina de estados empleada para la selección de las secuencias.

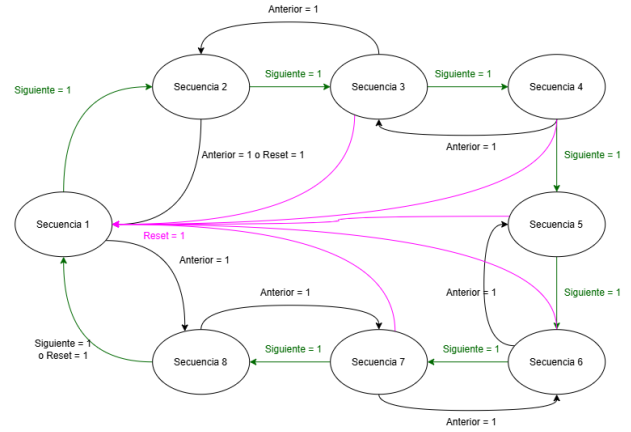


Fig. 4: Máquina de estados del sistema de secuencias luminosas.

VII. RESULTADOS

A. Resultados de la lavadora automática

El programa fue ensamblado en Microchip Studio y posteriormente simulado en PICSIMLab. Se verificó el funcionamiento de la selección de carga, los sensores de puerta y llenado, la temporización de los diferentes estados, los indicadores LED y las señales utilizadas para representar el sentido de giro del motor.

Durante las pruebas se comprobó la ejecución de los cinco ciclos de lavado y la transición automática hacia centrifugado y secado. También se verificó que los tiempos de giro y pausa varían correctamente según la carga ligera, media o pesada seleccionada.

La apertura de la puerta durante el proceso produjo correctamente la detención del motor y la pausa de la temporización. Una vez restablecida la condición de puerta cerrada, el programa continuó desde el mismo estado sin reiniciar el procedimiento.

La simulación final presentó un funcionamiento estable y permitió comprobar el recorrido completo desde el estado inicial hasta la indicación de fin del proceso.

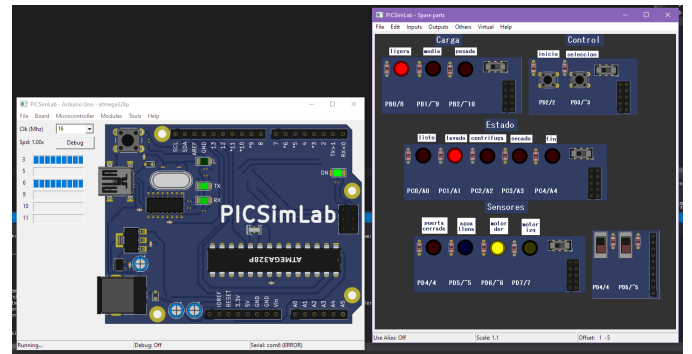


Fig. 5: Simulación de la lavadora automática en PICSIMLab.

B. Resultados del contador digital

El programa fue ensamblado en Microchip Studio y posteriormente simulado en PICSIMLab. Durante las pruebas se verificó el funcionamiento de los tres pulsadores, los límites establecidos para el contador y la correcta representación de los valores en el display de 7 segmentos.

El contador inició correctamente en cero y respondió a cada pulsación aumentando o disminuyendo una unidad. Al alcanzar los límites de 0 y 9, el valor permaneció sin cambios ante una solicitud

fuera del rango establecido. El pulsador de reinicio permitió regresar directamente al valor inicial desde cualquier posición.

Inicialmente se observó que el rebote producido al liberar los pulsadores podía generar una segunda modificación del contador. Para corregir este comportamiento se utilizó el Timer0 junto con una condición de bloqueo hasta detectar una liberación estable del pulsador. Luego de esta modificación se obtuvo una única acción por cada pulsación.

TABLE I: Pruebas realizadas sobre el contador digital.

Prueba	Resultado
Encendido del sistema	Contador inicia en 0
Incremento	Aumenta una unidad hasta 9
Decremento	Disminuye una unidad hasta 0
Reinicio	Regresa directamente a 0
Antirrebote	Una acción por pulsación

La simulación final presentó un comportamiento estable y permitió comprobar el cumplimiento de las funciones requeridas para el contador digital.

C. Resultados del sistema de secuencias

El programa fue ensamblado en Microchip Studio y simulado en PICSIMLab. Se verificó la ejecución continua de las ocho secuencias luminosas, observándose patrones diferenciables y estables en los ocho LEDs.

Los pulsadores permitieron avanzar y retroceder correctamente entre las secuencias, incluyendo las transiciones entre los extremos del rango. El pulsador de reinicio permitió regresar directamente a la secuencia 1 desde cualquier otra secuencia. También se comprobó que el cambio podía realizarse durante la ejecución de una animación, sin necesidad de esperar a que esta completara su ciclo.

El uso del Timer0 permitió mantener un intervalo regular entre los cambios de los patrones y conservar el mecanismo de antirrebote utilizado para los pulsadores.

TABLE II: Pruebas realizadas sobre el sistema de secuencias.

Prueba	Resultado
Secuencias 1-8	Ejecución continua y estable
Siguiente	Avanza correctamente
Anterior	Retrocede correctamente
Reinicio	Regresa a la secuencia 1
Cambio durante ejecución	Respuesta inmediata
Antirrebote	Una acción por pulsación

La simulación final permitió comprobar el funcionamiento conjunto de las entradas, la temporización y los ocho patrones luminosos implementados.

VIII. CONCLUSIONES

El desarrollo del laboratorio permitió aplicar los principales recursos del ATmega328P mediante programación en lenguaje ensamblador. A través de los diferentes sistemas implementados se trabajó con la configuración directa de puertos de entrada y salida, registros, interrupciones, temporizadores, direccionamiento de memoria y operaciones lógicas, permitiendo comprender con mayor profundidad la interacción entre el software y los periféricos del microcontrolador.

La utilización de interrupciones y del Timer0 permitió controlar las entradas y las temporizaciones sin depender de retardos bloqueantes. Esto resultó especialmente útil en la lavadora automática y en el sistema de secuencias, donde era necesario mantener la capacidad de responder ante cambios durante la ejecución. En el contador digital, las pruebas permitieron detectar problemas producidos por el rebote de los pulsadores, los cuales fueron corregidos mediante un mecanismo de antirrebote basado en el temporizador. Además, el uso de una tabla de búsqueda almacenada en SRAM permitió simplificar

la conversión de los valores del contador a los patrones del display de 7 segmentos.

Finalmente, las simulaciones realizadas en PICSIMLab permitieron verificar el funcionamiento de los programas y comprobar el cumplimiento de las condiciones planteadas para cada sistema. El laboratorio permitió integrar los conceptos trabajados durante el curso y evidenció la importancia de organizar el programa de forma modular y seleccionar adecuadamente los recursos del microcontrolador según las necesidades de cada aplicación.

IX. RECURSOS COMPLEMENTARIOS

Carpeta en Google Drive de Evidencia Experimental
Repositorio en GitHub

X. BIBLIOGRAFÍA

REFERENCES

- [1] ATmega328P Datasheet

APPENDIX A

DISEÑO DE LA LAVADORA AUTOMÁTICA

A. Descripción general

El sistema implementó el control de una lavadora automática mediante el microcontrolador ATmega328P. El funcionamiento se organizó mediante una máquina de estados encargada de controlar las etapas de lavado, centrifugado y secado.

La carga puede seleccionarse mediante un único pulsador, recorriendo cíclicamente los valores ligera, media y pesada. La carga seleccionada permanecerá indicada mediante tres LEDs y no podrá modificarse una vez iniciado el proceso.

El sistema cuenta además con un pulsador de inicio, un sensor de puerta cerrada, un sensor de llenado de agua y dos señales para representar el sentido de giro del motor.

B. Asignación de pines

TABLE III: Asignación de pines de la lavadora automática.

Elemento	Arduino UNO	ATmega328P	Función
Inicio	D2	PD2	Entrada / INT0
Selección de carga	D3	PD3	Entrada / INT1
Puerta cerrada	D4	PD4	Entrada
Sensor de agua	D5	PD5	Entrada
Motor derecha	D6	PD6	Salida
Motor izquierda	D7	PD7	Salida
Carga ligera	D8	PB0	Salida
Carga media	D9	PB1	Salida
Carga pesada	D10	PB2	Salida
Listo	A0	PC0	Salida
Lavado	A1	PC1	Salida
Centrifugado	A2	PC2	Salida
Secado	A3	PC3	Salida
Fin	A4	PC4	Salida

Los pulsadores de inicio y selección se configuran con resistencias *pull-up* internas. Por lo tanto, las pulsaciones serán detectadas mediante flanco de bajada utilizando las interrupciones externas INT0 e INT1.

Los sensores de puerta y llenado también utilizan resistencias *pull-up* internas, considerando un nivel lógico bajo como condición activa.

C. Máquina de estados

Se utilizan los siguientes estados:

TABLE IV: Estados principales del sistema.

Estado	Función
LISTO	Espera selección de carga e inicio
ESPERA_PUERTA	Espera que la puerta se encuentre cerrada
ESPERA_LLENADO	Espera confirmación del sensor de agua
LAVADO_GIRO	Motor girando durante el lavado
LAVADO_PAUSA	Pausa entre giros de lavado
CENTRIFUGADO	Giro continuo de centrifugado
SECADO_DER	Giro hacia la derecha
SECADO_PAUSA	Pausa del secado
SECADO_IZQ	Giro hacia la izquierda
FIN	Proceso terminado

El procedimiento de lavado alterna entre LAVADO_GIRO y LAVADO_PAUSA hasta completar cinco repeticiones.

D. Tiempos según la carga

Se adopta la carga ligera como tiempo base. Para carga media se aplica una vez el incremento indicado por la consigna y para carga pesada se aplica dos veces.

TABLE V: Tiempos utilizados según la carga seleccionada.

Acción	Ligera	Media	Pesada
Lavado – giro	2 s	3 s	4 s
Lavado – pausa	1 s	2 s	3 s
Centrifugado	15 s	18 s	21 s
Secado – derecha	5 s	7 s	9 s
Secado – pausa	3 s	5 s	7 s
Secado – izquierda	5 s	7 s	9 s

E. Temporización e interrupciones

Se utiliza el Timer0 en modo CTC generando una interrupción cada 1 ms, siguiendo el mismo criterio utilizado en el contador digital.

Esta base temporal tendrá dos funciones:

- realizar el antirrebote de los pulsadores;
- generar unidades de 100 ms utilizadas para la temporización de los estados.

Las interrupciones INT0 e INT1 atenderán respectivamente los pulsadores de inicio y selección de carga. El antirrebote utiliza las variables *boton_activo* y *debounce*, considerando nuevamente 20 ms de estabilidad antes de habilitar una nueva pulsación.

Los sensores de puerta y llenado son consultados mediante *polling* desde el programa principal.

F. Control del motor

El motor es representado mediante dos salidas digitales:

TABLE VI: Control lógico del motor.

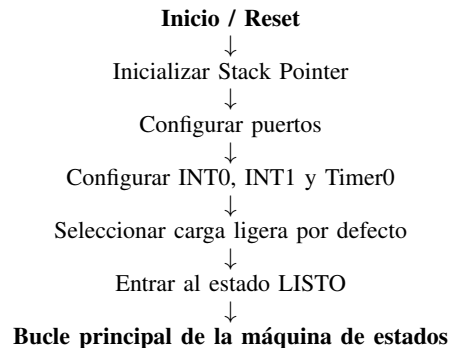
Derecha	Izquierda	Estado
0	0	Motor detenido
1	0	Giro hacia la derecha
0	1	Giro hacia la izquierda

G. Registros previstos

TABLE VII: Registros principales utilizados.

Registro	Uso
r16	Temporal
r17	Estado actual
r18	Contador de antirrebote
r19	Pulsador activo
r20	Carga seleccionada
r21	Banderas de control
r22	Tiempo del estado actual
r23	Contador de ciclos de lavado
r24	Base temporal

H. Flujo general del programa



Durante el bucle principal se consultan los sensores, se procesa el estado actual y se actualizan los LEDs y el motor. Las interrupciones de los pulsadores registran los eventos de inicio o selección, mientras que Timer0 proporciona la referencia temporal.

Como condición de seguridad, el lavado no comenzará hasta que la puerta se encuentre cerrada y el sensor de llenado se encuentre activo. Si la puerta se abre durante cualquiera de los estados del proceso, el motor se detiene y la temporización queda pausada hasta que la puerta vuelva a cerrarse.

APPENDIX B

DISEÑO DEL CONTADOR DIGITAL

A. Descripción general

Se implementó un contador digital de un dígito utilizando el microcontrolador ATmega328P. El valor del contador se encuentra limitado entre 0 y 9 y se visualiza permanentemente mediante un display de 7 segmentos.

El sistema dispone de tres pulsadores: uno para incrementar el contador, otro para decrementarlo y un tercero para reiniciarlo a cero. La detección de los pulsadores se realiza mediante interrupciones, mientras que el Timer0 se utiliza como referencia temporal para implementar el mecanismo de antirrebote.

B. Asignación de pines

TABLE VIII: Asignación de pines del contador digital.

Elemento	Arduino UNO	ATmega328P	Función
Incrementar	D2	PD2	Entrada / INT0
Decrementar	D3	PD3	Entrada / INT1
Reset	D4	PD4	Entrada / PCINT20
Segmento A	D8	PB0	Salida
Segmento B	D9	PB1	Salida
Segmento C	D10	PB2	Salida
Segmento D	D11	PB3	Salida
Segmento E	D12	PB4	Salida
Segmento F	D13	PB5	Salida
Segmento G	D7	PD7	Salida

Los tres pulsadores fueron configurados como entradas con resistencias *pull-up* internas. Por esta razón, el estado normal corresponde a un nivel lógico alto y una pulsación se detecta mediante un nivel lógico bajo.

Los pulsadores de incremento y decremento utilizan las interrupciones externas INT0 e INT1 respectivamente, configuradas para detectar un flanco de bajada. El pulsador de reinicio se conecta a PD4 y utiliza la interrupción por cambio de pin PCINT20.

C. Tabla de búsqueda del display

La conversión entre el valor del contador y los segmentos que deben encenderse se realiza mediante una tabla de búsqueda o LUT (*Look Up Table*).

Se utilizó un display de cátodo común, donde un nivel lógico alto enciende el segmento correspondiente. Los patrones se encuentran organizados según el orden:

G F E D C B A

TABLE IX: Patrones utilizados en la LUT del display.

Número	GFEDCBA	Hexadecimal
0	0111111	0x3F
1	0000110	0x06
2	1011011	0x5B
3	1001111	0x4F
4	1100110	0x66
5	1101101	0x6D
6	1111101	0x7D
7	0000111	0x07
8	1111111	0x7F
9	1101111	0x6F

Los diez patrones se almacenan consecutivamente en memoria SRAM a partir de la dirección 0x0100. Para acceder al patrón correspondiente se utiliza el registro Y como puntero. El valor actual

del contador se suma a la dirección base de la tabla y posteriormente se utiliza la instrucción LD para cargar el patrón.

Los segmentos A–F son enviados mediante PB0–PB5. El bit correspondiente al segmento G se separa del patrón y se envía mediante PD7.

D. Temporización y antirrebote

El Timer0 fue configurado en modo CTC (*Clear Timer on Compare Match*) con un prescaler de 64 y un valor de comparación:

$$OCR0A = 249$$

Considerando una frecuencia de reloj de 16 MHz, esta configuración genera una interrupción cada 1 ms.

El antirrebote se implementó utilizando los registros `boton_activo` y `debounce`. Cuando se detecta una pulsación válida, se identifica el pulsador activo y se carga un valor de 20 en el contador de antirrebote.

La modificación del contador se realiza inmediatamente dentro de la rutina de interrupción correspondiente. Mientras el pulsador continúe presionado, el contador de antirrebote se mantiene en 20. Una vez detectada su liberación, Timer0 decrementa este valor cada 1 ms. El pulsador vuelve a habilitarse únicamente después de permanecer liberado de forma estable durante 20 ms.

Este mecanismo evita que una misma pulsación sea interpretada varias veces sin introducir retardos bloqueantes dentro del programa.

E. Funcionamiento de las interrupciones

La interrupción INT0 se utiliza para incrementar el contador. Antes de realizar la operación se comprueba que no exista otro pulsador activo y que el valor sea menor que 9.

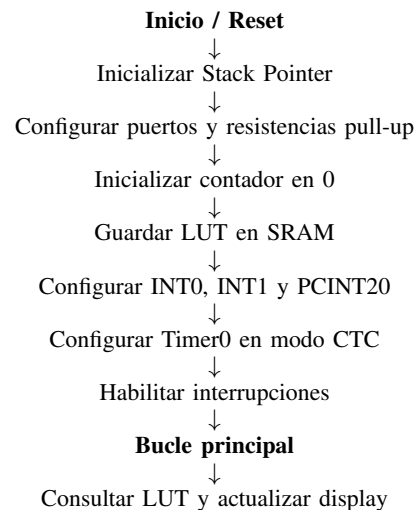
La interrupción INT1 se utiliza para decrementar el contador. La operación solamente se realiza cuando no existe otro pulsador activo y el contador es mayor que 0.

El pulsador de reinicio utiliza PCINT20. Debido a que una interrupción por cambio de pin se produce tanto al presionar como al liberar el pulsador, dentro de la rutina se comprueba el nivel de PD4 y solamente se ejecuta el reinicio cuando el pin se encuentra en nivel lógico bajo.

En todas las rutinas de interrupción se almacena temporalmente el registro SREG en el Stack antes de realizar las operaciones y se restaura antes de ejecutar RETI.

F. Flujo general del programa

El funcionamiento general del programa puede representarse mediante el siguiente flujo:



El bucle principal se encarga únicamente de consultar el patrón correspondiente al valor actual del contador y actualizar continuamente el display. Las modificaciones del contador son realizadas de forma asíncrona mediante las rutinas de interrupción de los tres pulsadores.

G. Registros principales utilizados

TABLE X: Registros utilizados en el contador digital.

Registro	Uso
r16	Registro temporal
r18	Contador de antirrebote
r19	Identificación del pulsador activo
r20	Valor actual del contador
r21	Patrón del display
r28:r29	Puntero Y para acceso a la LUT en SRAM

APPENDIX C

DISEÑO DEL SISTEMA DE SECUENCIAS CON 8 LEDs

A. Descripción general

Se implementó un sistema compuesto por ocho LEDs controlados mediante el microcontrolador ATmega328P, capaz de ejecutar ocho secuencias luminosas diferentes. La secuencia seleccionada se ejecuta de forma continua hasta que el usuario selecciona otra mediante tres pulsadores.

Los pulsadores permiten avanzar a la siguiente secuencia, retroceder a la anterior o regresar directamente a la secuencia 1. La selección se implementó de forma circular, por lo que al avanzar desde la secuencia 8 se regresa a la secuencia 1, mientras que al retroceder desde la secuencia 1 se pasa a la secuencia 8.

La detección de los pulsadores se realiza mediante interrupciones. El Timer0 se utiliza como base temporal tanto para el antirrebote como para controlar el intervalo entre los diferentes pasos de las animaciones.

B. Asignación de pines

La distribución de los pulsadores mantiene los mismos pines utilizados en el contador digital.

TABLE XI: Asignación de pines del sistema de secuencias.

Elemento	Arduino UNO	ATmega328P	Función
Siguiente	D2	PD2	Entrada / INT0
Anterior	D3	PD3	Entrada / INT1
Reset	D4	PD4	Entrada / PCINT20
LED 1	D5	PD5	Salida
LED 2	D6	PD6	Salida
LED 3	D7	PD7	Salida
LED 4	D8	PB0	Salida
LED 5	D9	PB1	Salida
LED 6	D10	PB2	Salida
LED 7	D11	PB3	Salida
LED 8	D12	PB4	Salida

Los tres pulsadores fueron configurados como entradas con resistencias *pull-up* internas. Por esta razón, el estado normal corresponde a un nivel lógico alto y las pulsaciones se detectan mediante un nivel lógico bajo.

Los pulsadores Siguiente y Anterior utilizan las interrupciones externas INT0 e INT1 respectivamente, configuradas por flanco de bajada. El pulsador Reset utiliza la interrupción por cambio de pin PCINT20.

C. Representación de los LEDs

Los ocho LEDs se representan internamente mediante un único patrón de 8 bits:

LED8 LED7 LED6 LED5 LED4 LED3 LED2 LED1

Los tres bits menos significativos corresponden a LED1–LED3 y son enviados a PD5–PD7. Los cinco bits restantes corresponden a LED4–LED8 y son enviados a PB0–PB4.

La rutina `mostrar_patron` realiza esta distribución mediante máscaras y desplazamientos. Al escribir sobre PORTD también mantiene activadas las resistencias *pull-up* de PD2–PD4.

D. Secuencias implementadas

Las ocho secuencias fueron desarrolladas principalmente mediante operaciones binarias sobre el patrón de LEDs, evitando almacenar listas extensas de valores constantes.

Cada rutina modifica solamente un paso de la animación y posteriormente retorna al programa principal. De esta forma no se utilizan

TABLE XII: Secuencias luminosas implementadas.

Sec.	Comportamiento	Operaciones
1	LED hacia la izquierda	LSL
2	LED hacia la derecha	LSR
3	Movimiento de ida y vuelta	LSL, LSR
4	Llenado y vaciado progresivo	ROL, ROR
5	LEDs alternados	EOR
6	Extremos hacia el centro y regreso	LSL, LSR, OR
7	Alternancia entre ambas mitades	SWAP
8	Conteo de 4 bits repetido	INC, ANDI, SWAP, OR

retardos bloqueantes y el sistema puede responder a un cambio de secuencia mientras una animación se encuentra en ejecución.

Las secuencias 3, 4 y 6 utilizan además el registro `estado_seq` para recordar la dirección o etapa actual de la animación.

E. Temporización

El Timer0 fue configurado en modo CTC (*Clear Timer on Compare Match*) con un prescaler de 64 y un valor de comparación:

$$OCR0A = 249$$

Considerando una frecuencia de reloj de 16 MHz, esta configuración genera una interrupción cada 1 ms.

El registro `anim_timer` se inicializa con un valor de 200 y se decrementa en cada interrupción del Timer0. Al llegar a cero se activa la bandera `frame_ready` y se reinicia el contador en 200, obteniendo un intervalo de aproximadamente 200 ms entre los pasos de cada secuencia.

El programa principal únicamente actualiza la animación cuando `frame_ready` se encuentra activa. Esto permite mantener la ejecución del programa libre de retardos bloqueantes.

F. Antirrebote de los pulsadores

El mismo Timer0 se utiliza como referencia temporal para implementar el antirrebote de los tres pulsadores.

Cuando se detecta una pulsación válida, el registro `boton_activo` identifica cuál de los tres pulsadores produjo el evento y el registro `debounce` se inicializa en 20.

Mientras el pulsador continúe presionado, este valor se mantiene en 20. Una vez detectada su liberación, el Timer0 decrementa el contador cada 1 ms. El sistema permite aceptar una nueva pulsación únicamente después de que el botón haya permanecido liberado de forma estable durante 20 ms.

En el caso del pulsador Reset, al utilizar una interrupción por cambio de pin, se verifica además el nivel de PD4 para diferenciar la pulsación de la liberación.

G. Selección y reinicio de secuencias

Las rutinas de interrupción de los pulsadores modifican directamente el registro `secuencia`.

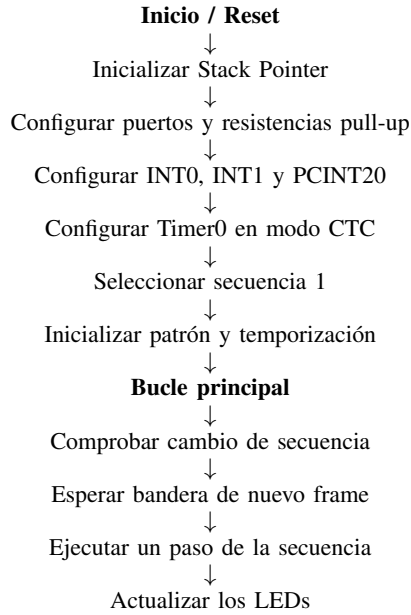
- **Siguiente:** incrementa la secuencia activa. Si se encuentra en la secuencia 8, regresa a la secuencia 1.
- **Anterior:** decrementa la secuencia activa. Si se encuentra en la secuencia 1, pasa a la secuencia 8.
- **Reset:** selecciona directamente la secuencia 1.

Cuando se produce un cambio se activa la bandera `reiniciar`. El bucle principal detecta esta condición y ejecuta la rutina `inicializar_secuencia`, encargada de restablecer el patrón inicial, el estado interno y la temporización correspondiente.

De esta forma, el cambio entre secuencias puede realizarse inmediatamente sin esperar que la animación anterior complete su ciclo.

H. Flujo general del programa

El funcionamiento general puede representarse mediante el siguiente flujo:



Las rutinas de interrupción se encargan de registrar los cambios solicitados por los pulsadores, mientras que Timer0 genera la referencia temporal. El bucle principal ejecuta únicamente un paso de la animación cuando corresponde y actualiza posteriormente las salidas.

I. Registros principales utilizados

TABLE XIII: Registros principales utilizados en el sistema de secuencias.

Registro	Uso
r16	Registro temporal
r18	Contador de antirrebote
r19	Identificación del pulsador activo
r20	Secuencia seleccionada
r21	Patrón actual de los LEDs
r22	Estado interno de la secuencia
r23	Temporización entre pasos
r24	Bandera de nuevo frame
r25	Bandera de reinicio de secuencia